

第三章

通用数据处理指令

目录

1. 数据传送类指令
2. 算术运算类指令
3. 位操作类指令

学习指令的注意事项

- **指令的功能**——该指令能够实现何种操作。通常指令助记符就是指令功能的英文单词或其缩写形式
- **指令支持的寻址方式**——该指令中的操作数可以采用何种寻址方式
- **指令对标志的影响**——该指令执行后是否对各个标志位有影响，以及如何影响
- **其他方面**——该指令其他需要特别注意的地方，如指令执行时的约定设置、必须预置的参数、隐含使用的寄存器等

汇编语言程序设计

MOV指令

数据传送类指令

➤ 数据传送

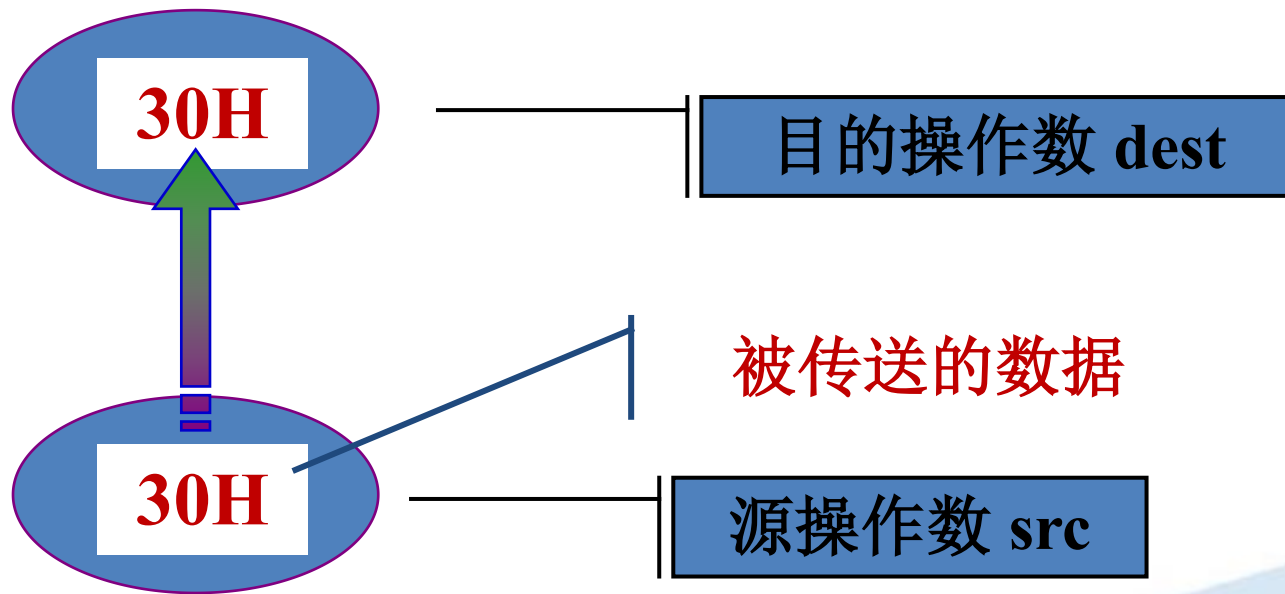
- ▶ 把数据从一个位置传送到另一个位置
- ▶ 计算机中最基本的操作
- ▶ 程序设计中最常使用的指令

➤ 除标志寄存器传送指令外，均不影响状态标志

MOV XCHG PUSH POP LEA XLAT

MOV指令的功能

➤ 提供方便灵活的通用数据传送操作



传送指令MOV (move)

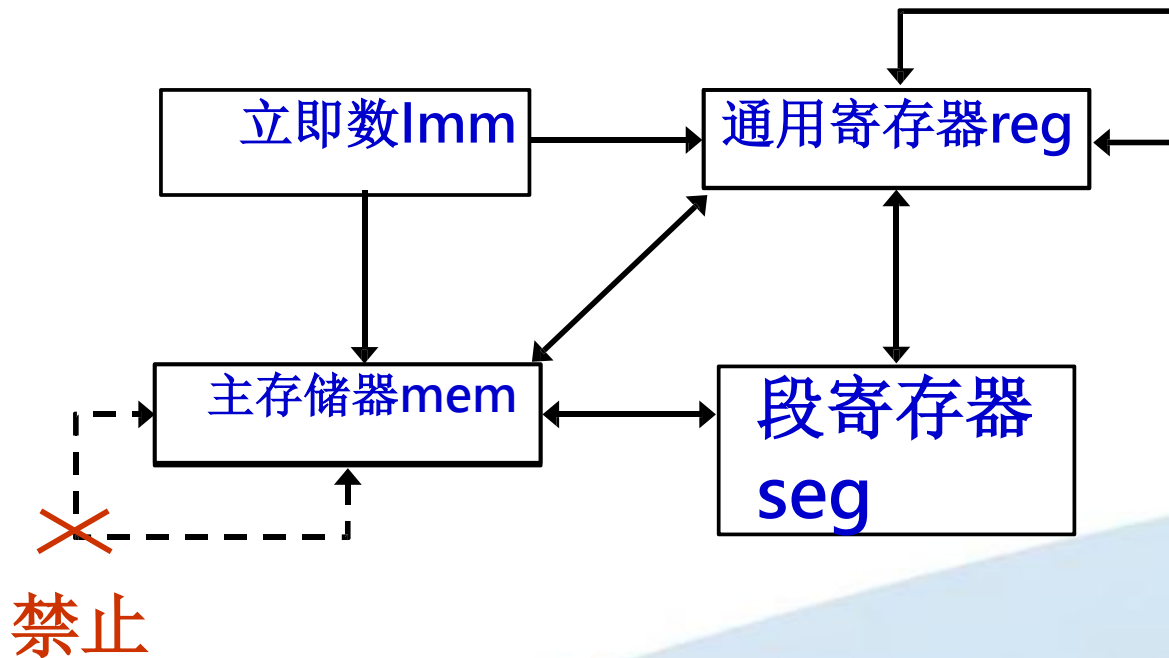
➤ 把一个字节、字或双字的操作数
从源位置传送至目的位置

MOV reg/mem,imm

MOV reg/mem/seg,reg

MOV reg/seg,mem

MOV r16/m16,seg



r16/m16:16位段寄存器

IA-32指令支持3种数据长度

➤ 8位（字节）数据，**byte**类型

```
mov al,200
```

➤ 16位（字）数据，**word**类型

```
mov ax,[ebx]
```

➤ 32位（双字）数据，**dword**类型

```
mov eax,dvar
```

32位通用寄存器:

EAX EBX ECX

16位通用寄存器:

AX BX CX DX

8位通用寄存器:

AH AL BH BL

16位段寄存器:

DS CS SS ES



立即数传送

➤ 寄存器 **reg** 为目的操作数

mov al,200

;8位立即数i8

mov ax,200

;16位立即数i16

mov eax,200

;32位立即数i32

➤ 存储器 **mem** 为目的操作数

mov bvar,byte ptr 200

;8位立即数i8

mov [ebx],word ptr 200

;16位立即数i16

mov [esi+8],dword ptr 200

;32位立即数i32

MOV reg/mem,imm



寄存器传送

➤ 寄存器 **reg** 为目的的操作数

mov al,ah

;8位通用寄存器r8

mov ax,bx

;16位通用寄存器r16

mov eax,edx

;32位通用寄存器r32

➤ 存储器 **mem** 为目的的操作数

mov bvar,cl

;8位通用寄存器r8

mov [ebx],cx

;16位通用寄存器r16

mov [esi+8],edi

;32位通用寄存器r32

➤ 段寄存器 **seg** 为目的的操作数

mov ds,bx

MOV reg/mem/seg,reg

存储器传送

➤ 寄存器 **reg** 为目的的操作数

mov dl,bvar

;8位存储器m8

mov dx,[ebx]

;16位存储器m16

mov edx,dvar[edi]

;32位存储器m32

➤ 段寄存器 **seg** 为目的的操作数

mov ds,wvar

;16位存储器m16

mov es,[ebx]

;16位存储器m16

mov ss,[ebp+8]

;16位存储器m16

MOV reg/seg,mem

(16位) 段寄存器传送

➤ 寄存器r16为目的操作数

```
mov ax,ds
```

```
mov dx,es
```

```
mov si,fs
```

```
mov di,gs
```

➤ 存储器m16为目的操作数

```
mov wvar,ds
```

```
mov [ebx],ss
```

```
mov [esi-8],cs
```

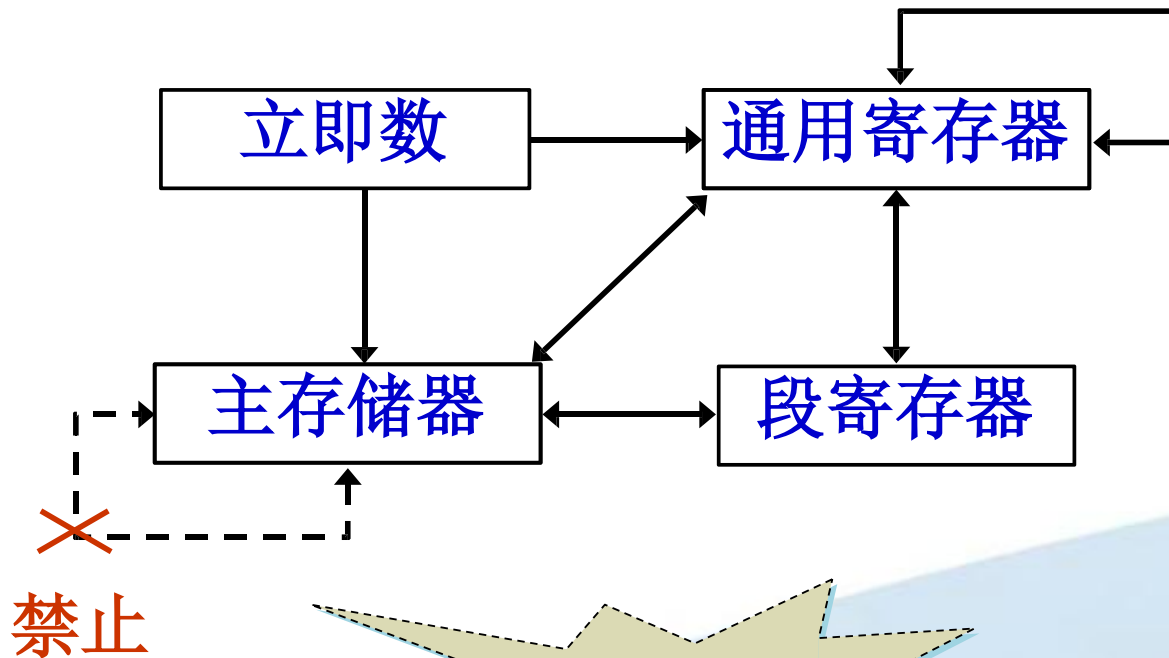
```
mov [ebp+8],cs
```

MOV r16/m16,seg

传送指令MOV (move)

- 把一个字节、字或双字的操作数
从源位置传送至目的位置

MOV reg/mem,imm
MOV reg/mem/seg,reg
MOV reg/seg,mem
MOV r16/m16,seg



并非任意传送！

XCHG指令

数据传送类指令-交换指令

——通用数据传送指令

2. 交换指令XCHG

功能：将源操作数和目的操作数内容交换

格式：

XCHG reg,reg/mem

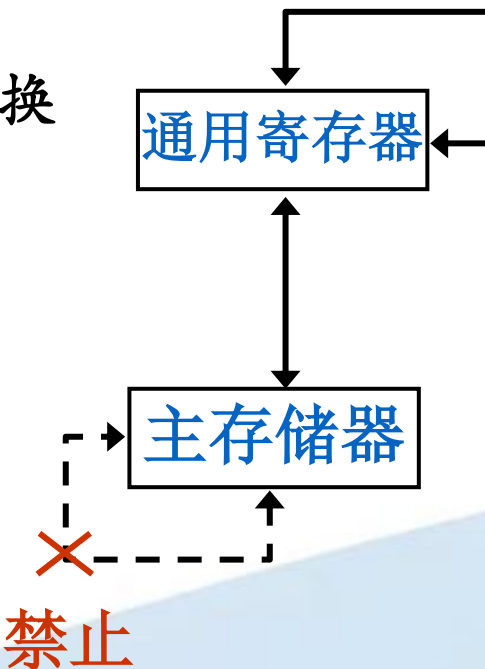
XCHG reg/mem,reg

例：

xchg esi,edi

xchg esi,[edi]

xchg al,var



注意:源和目的操作数长度一致



数据传送类指令

---通用数据传送指令

;实现两个存储器数据交换代码示例1

```
Include irvine32.inc
```

```
.data
```

```
wvar1 word 1234h
```

```
org 10h
```

```
wvar2 word 5678h
```

```
.code
```

```
main proc
```

```
mov ax,wvar1
```

```
xchg ax,wvar2
```

```
mov wvar1,ax
```

```
exit
```

```
main endp
```

```
end main
```

执行前内存数据

Address	Hex dump
00404000	34 12 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00404010	78 56 00 00 00 00 00 00 00 00 00 00 00 00 00 00

执行后内存数据

Address	Hex dump
00404000	78 56 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00404010	34 12 00 00 00 00 00 00 00 00 00 00 00 00 00 00



数据传送类指令

---通用数据传送指令

;实现两个存储器数据字节交换代码示例2

```
Include Irvine32.inc
```

```
.data
```

```
wvar1 word 1234h
```

```
org 10h
```

```
wvar2 word 5678h
```

```
.code
```

```
main proc
```

```
mov al,byte ptr wvar1
```

```
xchg al,byte ptr wvar2
```

```
mov byte ptr wvar1,al
```

```
mov al,byte ptr wvar1+1
```

```
xchg al,byte ptr wvar2+1
```

```
mov byte ptr wvar1+1,al
```

```
exit
```

```
main endp
```

```
end main
```

反汇编代码

```
MOV AL, BYTE PTR DS:[wvar1]  
XCHG BYTE PTR DS:[wvar2], AL  
MOV BYTE PTR DS:[wvar1], AL  
MOV AL, BYTE PTR DS:[404001]  
XCHG BYTE PTR DS:[404011], AL  
MOV BYTE PTR DS:[404001], AL
```

LEA指令

地址传送指令LEA (Load Effective Address)

➤地址传送指令获取存储器操作数的地址

LEA r16/r32,mem

;r16/r32 ← mem的有效地址EA (不需类型一致)

;IA-32使用32位地址, 保存于32位通用寄存器r32



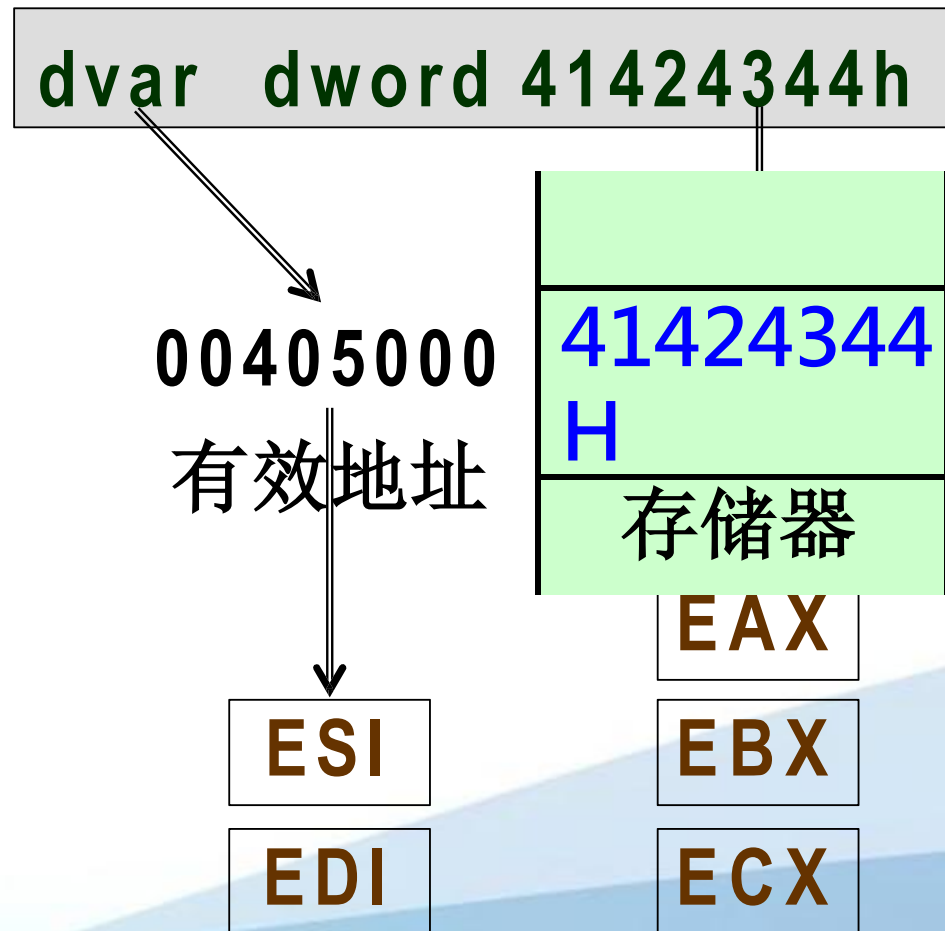
LEA 指令类似地址操作符OFFSET的作用

- LEA指令在**指令执行时**计算出偏移地址
 - ▶ OFFSET操作符在**汇编阶段**取得变量的偏移地址
- OFFSET无需在执行时计算、指令执行速度更快
 - ▶ LEA指令能获取汇编阶段无法确定的偏移地址

```
lea esi, var  
mov edi, offset var
```

地址传送程序

```
mov eax,dvar  
lea esi,dvar  
mov ebx,[esi]  
mov edi,offset dvar  
mov ecx,[edi]
```



用LEA指令实现运算功能

```
lea edx,[esi+edi*4+100h]
```

```
;EDX=ESI+EDI×4+100H
```

- LEA指令在计算地址时，可进行加和移位操作
 - 编译器将其用于实现加法，或者乘以2、4和8
- 不能用OFFSET操作符实现

本讲总结

- LEA指令获得存储器操作数的有效地址
 - 在LEA指令执行时计算地址
 - 对任何存储器寻址方式都可用
- 对存储器的直接寻址
 - 建议用OFFSET操作符在汇编阶段获得地址
- 对存储器的其他寻址
 - 只能使用LEA指令获得地址

汇编语言程序设计

XLAT指令

数据传送类指令-XLAT换码

---其他传送指令

格式: **XLAT**

功能: $AL \leftarrow [EBX + AL]$

即: 将EBX指定的缓冲区中AL指定的位移处的一个字节数据
取出赋给AL

换码指令执行前:

在主存建立一个字节量表格, 内含要目的代码

- 1) **表格首地址**存放于**EBX**
- 2) **AL**存放相对表格首地址的**位移量**

换码指令执行后:

将AL寄存器的内容转换为目标代码



数据传送类指令-XLAT

---其他传送指令

;换码程序代码1

使用XLAT指令

;数据段

num byte 6,7,7,8,3,0,0,0 ;被转换数字

tab byte '0123456789' ;代码表

;代码段，实现用数字查表换码为字符的功能

mov ecx,lengthof num

mov esi,offset num

mov ebx,offset tab

again: mov al,[esi]

xlat

call writechar

add esi,1

loop again

;EBX指向代码表

;AL=要转换的数字

;换码

;显示al寄存器中的字符

;指向下一个数字

;循环



数据传送类指令-XLAT

---其他传送指令

;换码程序代码2

不使用XLAT指令

;代码段

mov ecx,lengthof num

mov esi,offset num

mov ebx,offset tab

again: mov eax,0

mov al,[esi]

add eax,ebx

;EAX=EAX+EBX, 指向对应的字符

mov al,[eax]

call writechar

add esi,1

loop again

;EBX指向代码表

;EAX=0

;AL=要转换的数字

;换码

;显示

;指向下一个数字

;循环

汇编语言程序设计

PUSH 和 POP指令

PUSH和POP是堆栈操作指令

➤堆栈一个特殊的存储区域

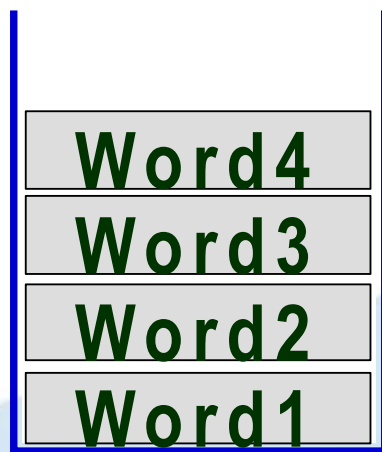
▶ 存取原则：先进后出**FILO**（**First In Last Out**）

也可称为：后进先出**LIFO**（**Last In First Out**）

▶ 具有两种基本的数据传输操作

• 数据压进堆栈 **PUSH**

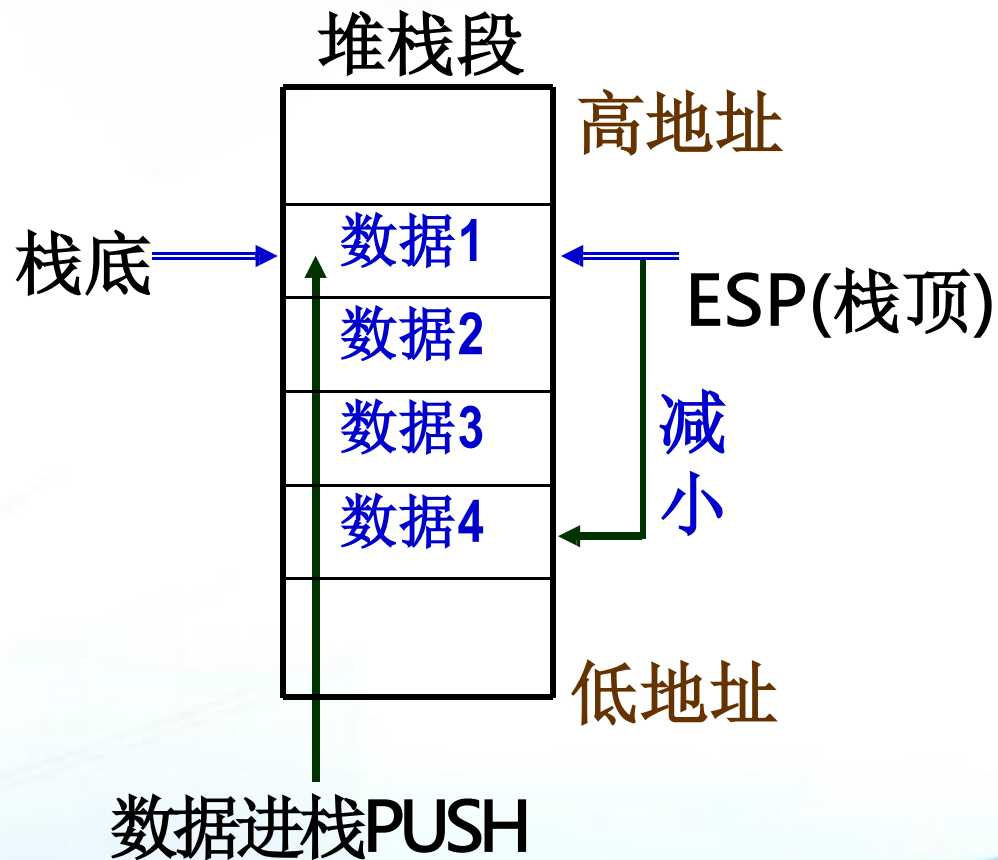
• 数据弹出堆栈 **POP**



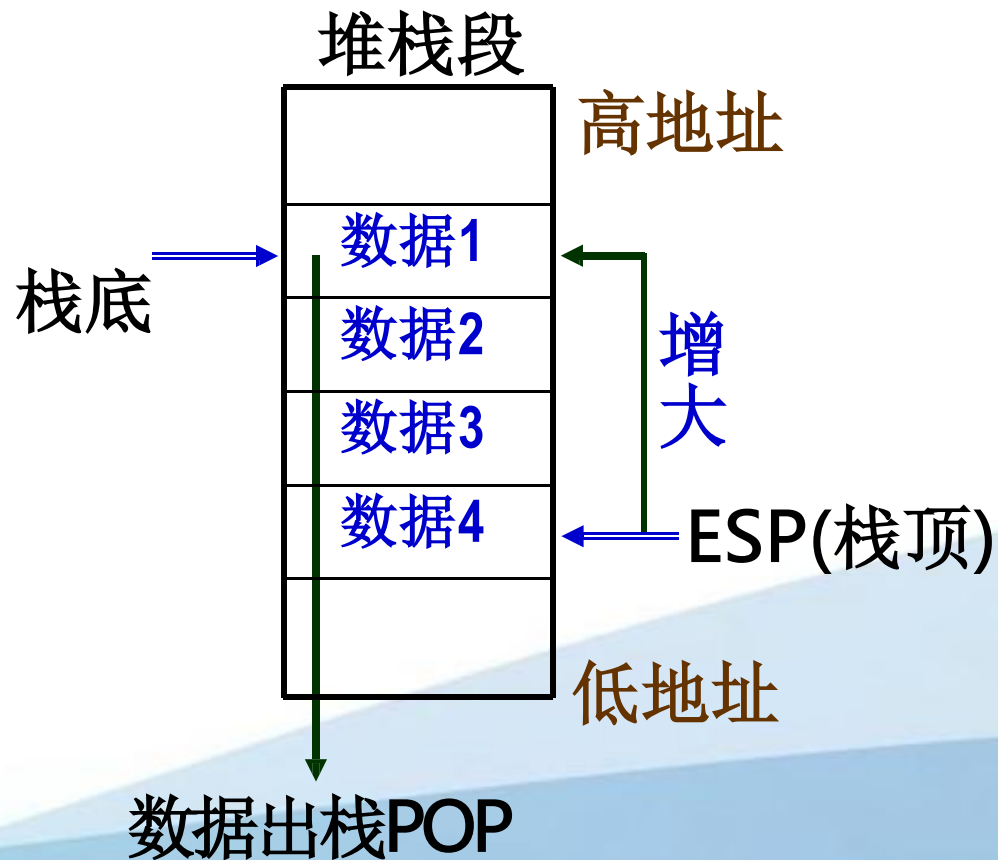
Stack

IA-32处理器的堆栈向下生长

数据压入堆栈、ESP逐渐减小



数据弹出堆栈、ESP逐渐增大



进栈指令PUSH

PUSH r16/m16/i16/seg

; ① $ESP = ESP - 2$ ② $SS:[ESP] = r16/m16/i16/seg$

PUSH r32/m32/i32

; ① $ESP = ESP - 4$ ② $SS:[ESP] = r32/m32/i32$

- ▶ 先将**ESP**减小作为当前栈顶
- ▶ 后将源操作数传送到当前栈顶
- 以字或双字为单位操作
 - ▶ 进栈字量数据前，**ESP**减2
 - ▶ 进栈双字量数据前，**ESP**减4

push eax

进栈PUSH操作

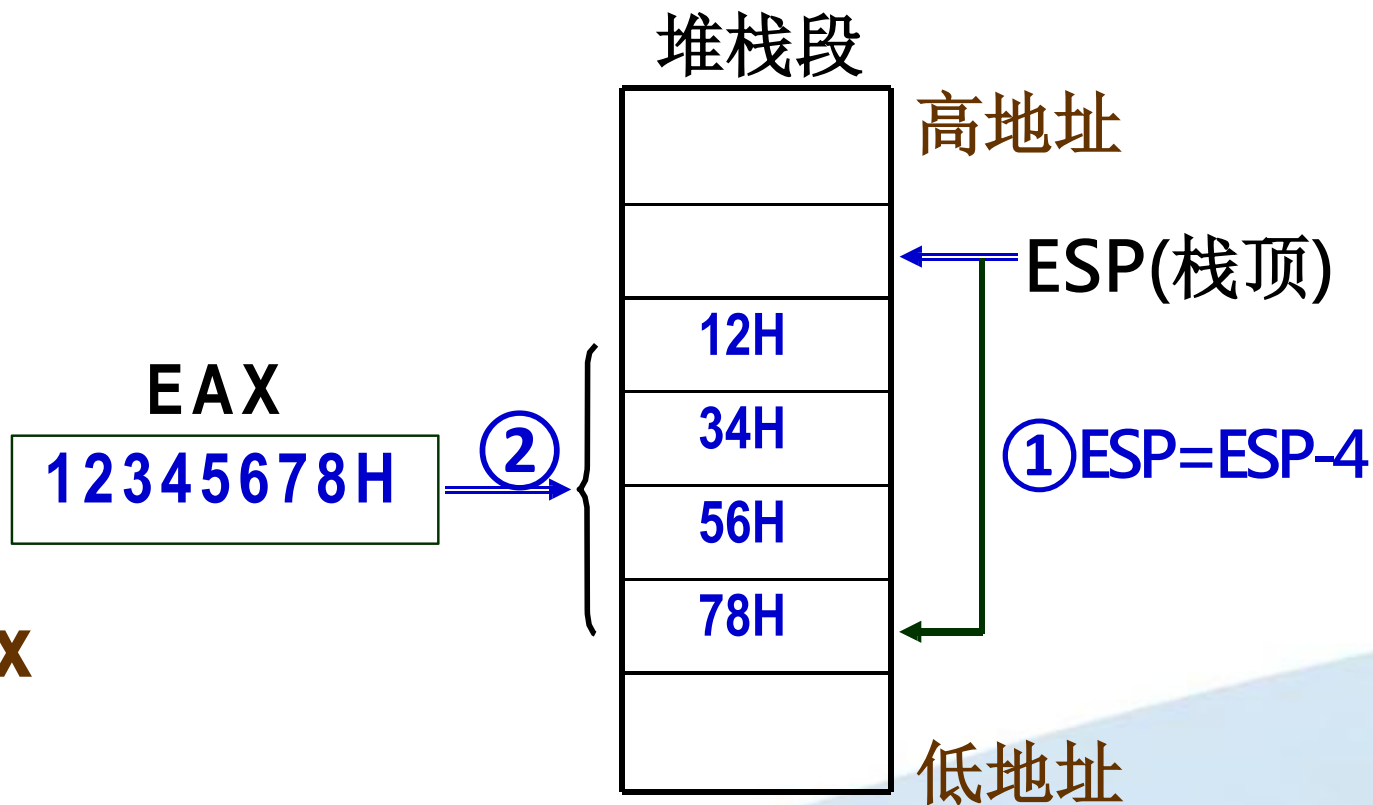
;进栈指令

push eax

;等同于如下2条指令

① **sub esp,4**

② **mov [esp],eax**



出栈指令POP

POP r16/m16/seg

; ① $r16/m16/seg = SS:[ESP]$ ② $ESP = ESP + 2$

POP r32/m32

; ① $r32/m32 = SS:[ESP]$ ② $ESP = ESP + 4$

- ▶ 先将栈顶数据传送到目的操作数
 - ▶ 后ESP增加作为当前栈顶
- 以字或双字为单位操作
- ▶ 出栈字量数据后，ESP加2
 - ▶ 出栈双字量数据后，ESP加4



pop eax

出栈POP操作

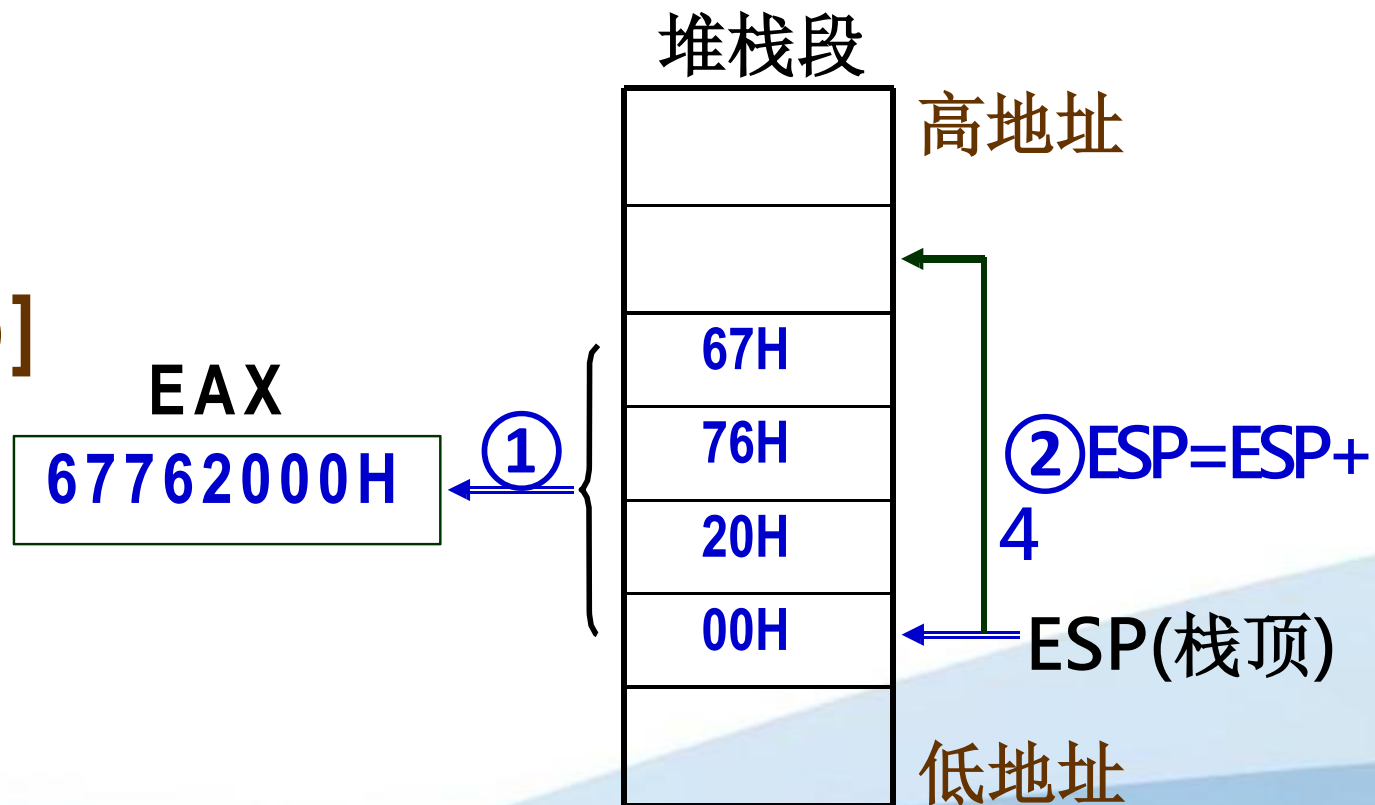
;进栈指令

pop eax

;等同于如下2条指令

① **mov eax,[esp]**

② **add esp,4**



堆栈操作程序—1

;数据段

ten = 10

dvar dword 67762000h,12345678h

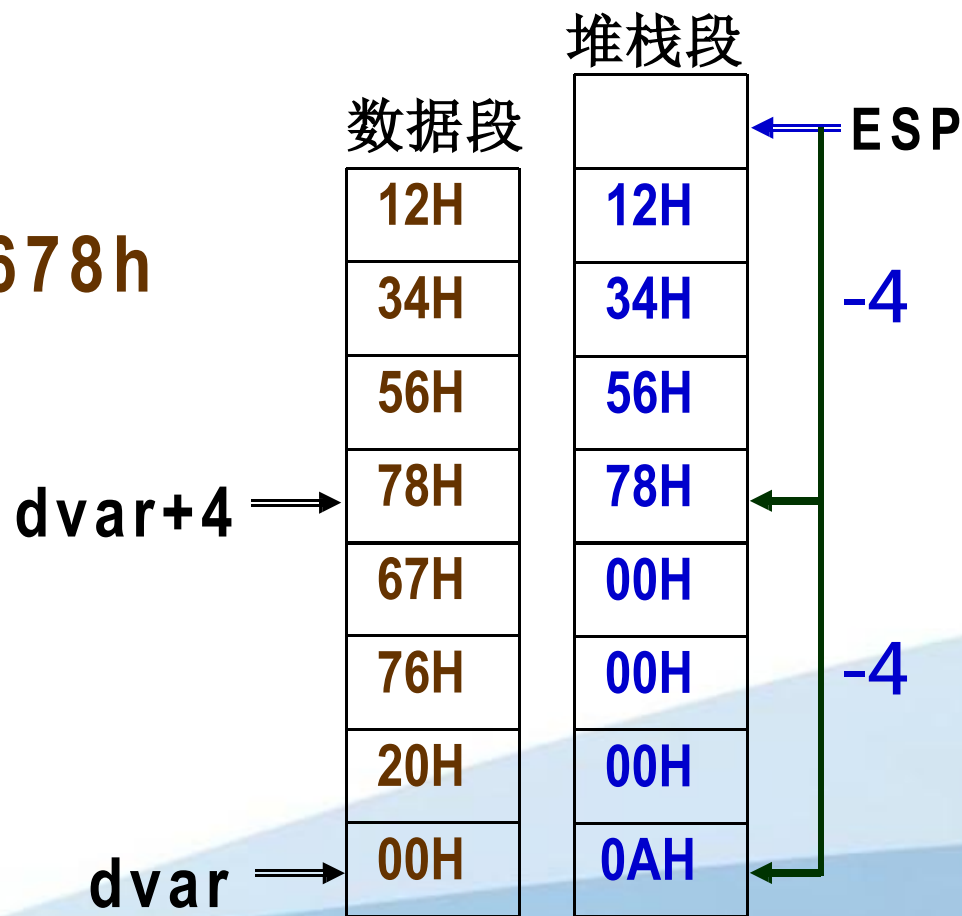
;代码段

mov eax,dvar+4

push eax

push dword ptr ten

;将立即数以双字量压入堆栈



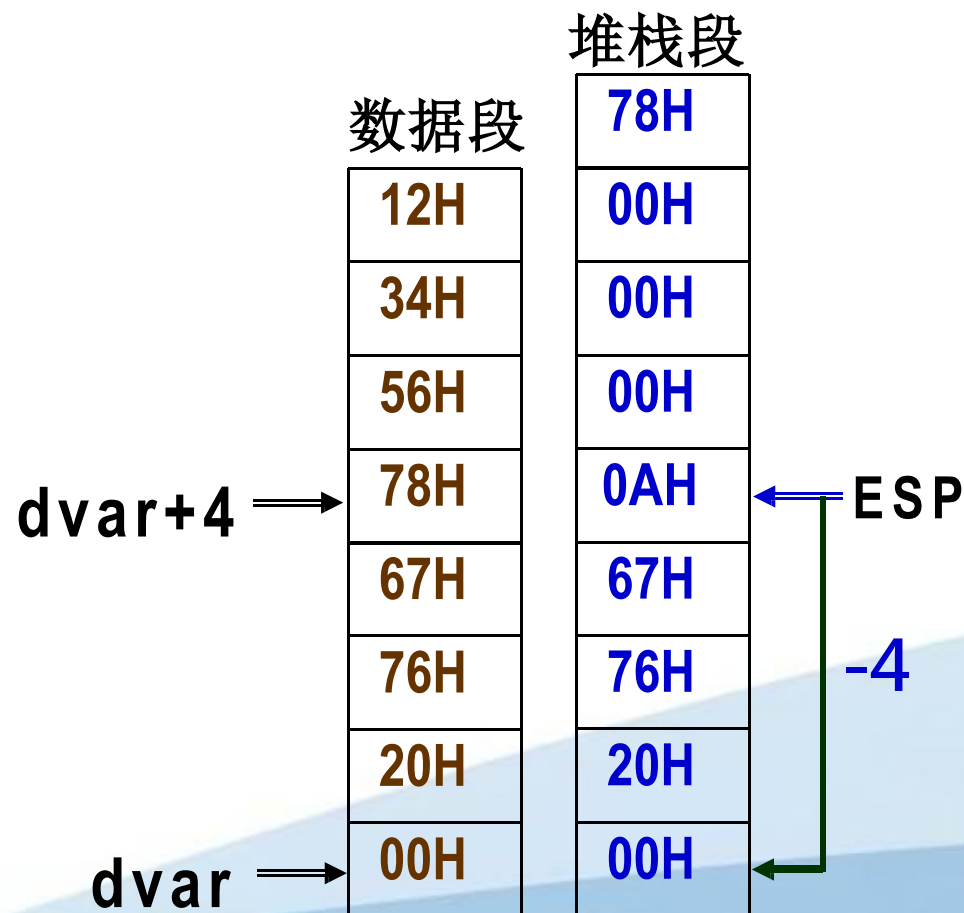
堆栈操作程序—2

push dvar

pop eax

;**栈顶数据弹出到EAX**

EAX = 67762000H





堆栈操作程序—3

pop dvar+4

;栈顶数据弹出到DVAR+4

mov ebx,dvar+4

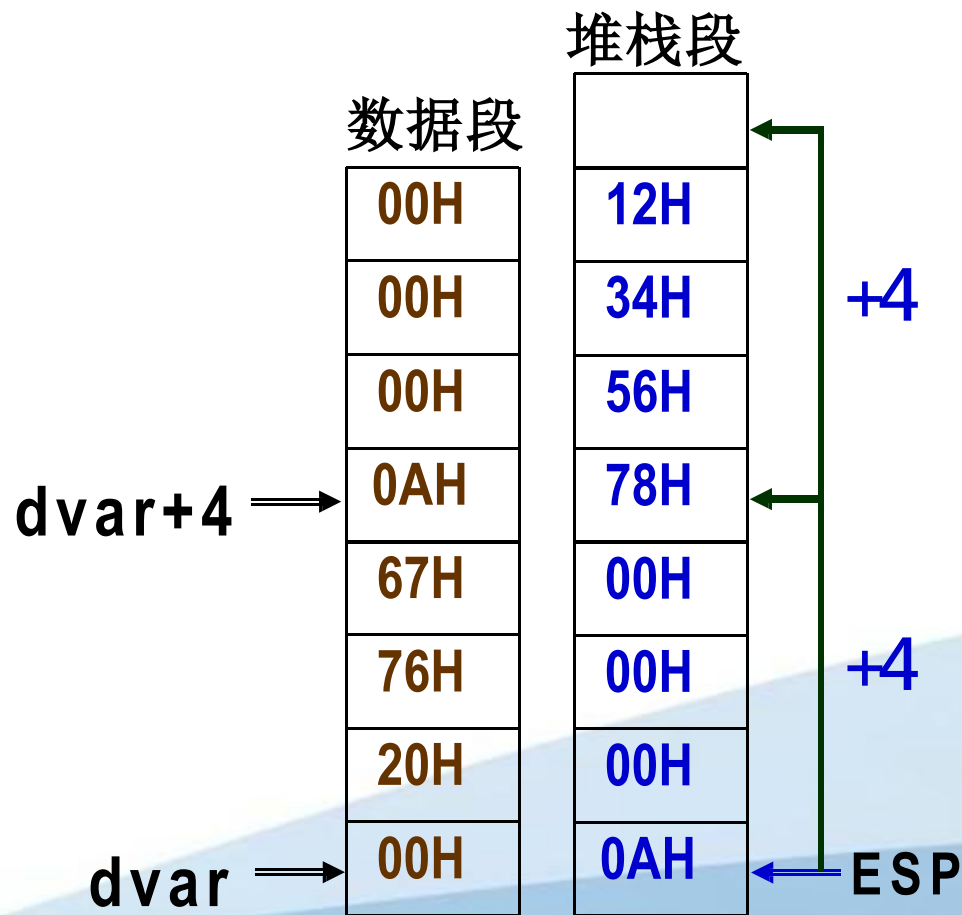
pop ecx

;栈顶数据弹出到ECX

EAX = 67762000H

EBX = 0000000AH

ECX = 12345678H



数据传送类指令---堆栈操作指令

堆栈说明:

1. 堆栈不可或缺, 被**很多指令使用**:
 - 堆栈操作指令
 - 子程序调用CALL和返回RET, 中断调用INT和返回IRET
 - 内部异常、外部中断等也利用堆栈
2. 堆栈可用**来临时存放数据**, 以便随时恢复它们
3. 常利用堆栈基址指针**EBP**, **随机读写堆栈数据**
4. 利用堆栈实现主、子程序间**传递参数**
5. 堆栈还常用于子程序的**寄存器保护**和恢复
6. 注意入栈和出栈的数据要成对, **保持堆栈平衡**

本讲总结

- 堆栈是一个后进先出LIFO存取原则的存储区域
 - ▶ 数据进栈（压入）使用**PUSH**指令
 - ▶ 数据出栈（弹出）使用**POP**指令
- 应用中要明确当前栈顶

